

Logiqa — ce qui est fait / ce qui reste

Revue du code après les corrections du stagiaire, par rapport au brief initial. Environ **70–80 %** des points critiques backend sont OK. Il reste : corriger le **double envoi OTP (2FA)** (double clic « Se connecter »), **brancher** le guard et l'interceptor, et corriger la **régression DROP TABLE** dans `init_db.py`.

Projet : Logiqa **Audit** : revue de code **Brief d'origine** : sécurité backend + frontend

~8 points **OK** **2 partiels** **4 à faire** (dont **1 critique** + **bug 2FA**)

Tableau récapitulatif

#	Point	Statut	Commentaire
1	Rôle forcé étudiant à l'inscription (API)	FAIT	<code>role = 'etudiant'</code> — ignore le body client
2	Anti-XSS champs texte (API)	FAIT	<code>contains_suspicious_html()</code> → 400
3	Mot de passe fort (API + front)	FAIT	≥ 8, lettre + chiffre
4	Rate limit login → 429	FAIT	5 essais / 5 min
—	Admin seedé dans <code>init_db.py</code>	FAIT	Compte admin créé au démarrage
5	Login Angular réel via API	FAIT	Appelle <code>/api/login</code> + OTP <code>/api/verify-otp</code>
7	Retirer onglet « administration »	FAIT	Reste étudiant / enseignant seulement
6	Auth guard sur <code>/dashboard</code>	PARTIEL	Fichier créé, pas branché dans <code>app.routes.ts</code>
8	Interceptor Bearer	PARTIEL	Fichier créé, pas enregistré dans <code>app.config.ts</code>
9	Captcha client	À FAIRE	Toujours en JS côté client (priorité basse)
10	URL API via <code>environment.ts</code>	À FAIRE	Toujours hardcodé <code>localhost:3000</code>
!	DROP TABLE à chaque démarrage	CRITIQUE	Efface toute la base à chaque <code>python app.py</code>
D	Code 2FA / OTP envoyé 2 fois	À FAIRE	Double clic « Se connecter » → <code>2x /api/login</code> → 2 emails

Bien fait — à conserver

Bravo sur le backend Les points critiques demandés dans le brief initial (rôle, XSS, password, rate limit, seed admin, login API, retrait administration) sont en place. Ne pas les casser.

Fait 1. Plus d'élévation de privilèges via `role`

Dans `backend/app.py` :

```

role = 'etudiant' # forcé — jamais depuis request.json

```

Même avec `"role": "admin"` en curl, le compte reste étudiant.

Fait 2–4. Validations + rate limit

- HTML suspect rejeté (contains_suspicious_html)
- Mot de passe validé (is_valid_password)
- Trop d'échecs login → HTTP 429

Fait 5. Login réel + OTP

login.ts appelle l'API. Le token est stocké après /api/verify-otp. Plus de login fake.

Fait 7. Onglet administration retiré

Type Role = 'etudiant' | 'enseignant' uniquement. L'admin reste uniquement via seed DB.

À faire maintenant (priorité)

Critique A. Supprimer les DROP TABLE dans init_db.py

Aujourd'hui, à chaque démarrage du serveur les tables sont détruites. C'est une régression grave.

Exemple — AVANT (mauvais) dans init_db.py :

```
# ■ NE PAS FAIRE — efface toute la base à chaque démarrage
cursor.execute("DROP TABLE IF EXISTS sessions")
cursor.execute("DROP TABLE IF EXISTS users")

cursor.execute("""
    CREATE TABLE users (
        id BIGINT AUTO_INCREMENT PRIMARY KEY,
        ...
    )
""")
```

Exemple — APRÈS (correct)

```
# ■ Créer seulement si absent — ne jamais DROP
cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id BIGINT AUTO_INCREMENT PRIMARY KEY,
        nom VARCHAR(100) NOT NULL,
        prenom VARCHAR(100) NOT NULL,
        email VARCHAR(150) NOT NULL UNIQUE,
        password VARCHAR(255) NOT NULL,
        telephone VARCHAR(50) NULL,
        role ENUM('admin', 'etudiant', 'enseignant') NOT NULL DEFAULT 'etudiant',
        last_login_at TIMESTAMP NULL,
        created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    )
""")

cursor.execute("""
    CREATE TABLE IF NOT EXISTS sessions (
        id BIGINT AUTO_INCREMENT PRIMARY KEY,
        user_id BIGINT NOT NULL,
        jti VARCHAR(100) NOT NULL UNIQUE,
        token_hash VARCHAR(255) NOT NULL,
        expires_at TIMESTAMP NOT NULL,
        revoked BOOLEAN NOT NULL DEFAULT FALSE,
        created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
    )
""")

# garder seed_admin(cursor, connection) après
```

1. Enlever les deux DROP TABLE.

2. Utiliser CREATE TABLE IF NOT EXISTS.

3. Garder seed_admin().

4. Test : redémarrer 2 fois → les comptes restent.

Haut B. Brancher authGuard sur /dashboard

Le fichier auth-guard.ts existe, mais n'est pas utilisé dans app.routes.ts.

Exemple — AVANT (mauvais) :

```
// app.routes.ts
{ path: 'dashboard', component: DashboardComponent }, // ■ public
```

Exemple — APRÈS (correct)

```
// app.routes.ts
import { authGuard } from './core/auth-guard';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  { path: 'inscription', component: InscriptionComponent },
  {
    path: 'dashboard',
    component: DashboardComponent,
    canActivate: [authGuard] // ■ protégé
  },
  { path: '**', redirectTo: '' }
];
```

Test manuel : ouvrir `http://localhost:4200/dashboard` sans être connecté → doit rediriger vers `/login`.

Haut C. Enregistrer l'interceptor HTTP

L'interceptor existe mais n'est pas enregistré → pas de header Bearer.

Exemple — AVANT (mauvais) dans app.config.ts :

```
provideHttpClient() // ■ interceptor ignoré
```

Exemple — APRÈS (correct)

```
// app.config.ts
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { authInterceptor } from './core/auth-interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])), // ■
    // ...
  ]
};
```

Exemple de header attendu sur les appels protégés :

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Bonus : sur réponse 401 → clear token + redirect login.

Haut D. Bug 2FA — le code OTP est envoyé 2 fois

Constat : il faut cliquer **2 fois** sur « Se connecter », et **2 emails** OTP arrivent. Logs serveur :

```
POST /api/login 200 (1er clic → 1er email OTP)
POST /api/login 200 (2e clic → 2e email OTP)
```

Où est l'erreur ?

- login.ts → validerConnexion() : pas de garde anti double-appel
- login.html → bouton « Se connecter » jamais désactivé
- app.py → /api/login renvoie un nouvel OTP à chaque succès (SMTP lent ~1–2 s)

Exemple — AVANT (mauvais) dans login.ts :

```
validerConnexion(): void {
  // ■ pas de isLoading → 2e clic possible pendant l'attente SMTP
  this.http.post('http://localhost:3000/api/login', {
    email: this.login,
    password: this.password
  }).subscribe({
    next: (response: any) => {
      this.pendingUserId = response.user_id;
      this.otpStep = true;
    },
    error: (err) => { /* ... */ }
  });
}
```

Exemple — APRÈS (correct) dans login.ts

```
isLoading = false;

validerConnexion(): void {
  if (this.isLoading) return; // ■ ignore le 2e clic

  if (this.captchaInput !== this.captchaCode) { /* ... */ return; }
  if (!this.login || !this.password) { /* ... */ return; }

  this.isLoading = true;

  this.http.post('http://localhost:3000/api/login', {
    email: this.login,
    password: this.password
  }).subscribe({
    next: (response: any) => {
      this.isLoading = false;
      this.pendingUserId = response.user_id;
      this.otpStep = true;
      this.errorVisible = false;
    },
    error: (err) => {
      this.isLoading = false;
      this.errorMessage = err.error?.error || 'Email ou mot de passe incorrect.';
      this.errorVisible = true;
      this.refreshCaptcha();
      this.captchaInput = '';
    }
  });
}
```

Exemple — APRÈS dans login.html

```
<button type="submit" class="btn-login" [disabled]="isLoading">
  {{ isLoading ? 'Envoi du code...' : 'Se connecter' }}
</button>
```

Exemple — APRÈS (recommandé) dans app.py /login

```
# Avant d'envoyer un nouvel email OTP :
existing = OTP_STORE.get(user['id'])
now = datetime.now(timezone.utc)

if existing and now < existing["expires_at"]:
    # OTP encore valide → ne pas renvoyer un 2e email tout de suite
    return jsonify({
        "success": True,
```

```

    "user_id": user['id'],
    "message": "Un code a déjà été envoyé. Vérifiez votre boîte mail."
  })

code = generate_otp()
OTP_STORE[user['id']] = {
  "code": code,
  "expires_at": now + timedelta(minutes=OTP_VALIDITY_MINUTES)
}
send_otp_email(user['email'], code)
return jsonify({"success": True, "user_id": user['id']})

```

Test attendu : 1 clic (même double-clic rapide) → **1 seul** email OTP.

À faire plus tard (priorité basse)

Moyen 9. Captcha 100 % client

Toujours généré en JS. Pas une vraie protection.

Exemple optionnel : retirer le bloc captcha du HTML et les champs captchaInput / captchaCode / refreshCaptcha() dans login.ts. Le rate limit API suffit pour l'instant.

Moyen 10. URL API hardcodée

Exemple — créer frontend/src/environments/environment.ts :

```

export const environment = {
  production: false,
  apiUrl: 'http://localhost:3000'
};

```

Puis dans login.ts / inscription.ts :

```

import { environment } from '../environments/environment';

this.http.post(`${environment.apiUrl}/api/login`, { ... })
this.http.post(`${environment.apiUrl}/api/register`, { ... })

```

Note rôles Le front envoie encore role: etudiant|enseignant — OK métier. L'API force déjà etudiant. Si les enseignants doivent s'inscrire, allowlist côté serveur :

```

ALLOWED_ROLES = {'etudiant', 'enseignant'}
role = data.get('role', 'etudiant')
if role not in ALLOWED_ROLES:
    role = 'etudiant'
# jamais 'admin' depuis le client

```

Ordre de travail (suite)

1. **Critique** : enlever DROP TABLE dans init_db.py.
2. **Bug 2FA** : bloquer le double clic dans login.ts / login.html (+ cooldown OTP côté API).
3. Brancher canActivate: [authGuard] sur dashboard.
4. Brancher withInterceptors([authInterceptor]) dans app.config.ts.
5. (Optionnel) environment.ts + retirer captcha client.
6. Retester la checklist ci-dessous.

Checklist de re-test (+ exemples curl)

Test	Résultat attendu
Redémarrer l'API 2 fois	Les users / admin sont toujours en base (pas de DROP)

Register avec role admin	Compte créé en etudiant
Password "1"	HTTP 400
Nom avec script HTML	HTTP 400
6+ mauvais mots de passe	HTTP 429
/dashboard sans token	Redirect /login
Login OK + OTP → appels API	Header Authorization: Bearer ... présent
1 clic / double-clic « Se connecter »	1 seul email OTP

Exemples de commandes pour tester l'API :

```
# 1) role=admin → doit rester etudiant
curl -s -X POST http://localhost:3000/api/register \
-H "Content-Type: application/json" \
-d '{"nom":"Test","prenom":"User","email":"t1@test.local","password":"Passw0rd1","role":"admin"}'

# 2) mot de passe trop faible → 400
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:3000/api/register \
-H "Content-Type: application/json" \
-d '{"nom":"A","prenom":"B","email":"t2@test.local","password":"1"}'

# 3) XSS dans nom → 400
curl -s -X POST http://localhost:3000/api/register \
-H "Content-Type: application/json" \
-d '{"nom":"<script>alert(1)</script>","prenom":"B","email":"t3@test.local","password":"Passw0rd1"}'

# 4) rate limit → après 5-6 échecs : 429
for i in 1 2 3 4 5 6; do
  curl -s -o /dev/null -w "%i:%{http_code} " -X POST http://localhost:3000/api/login \
  -H "Content-Type: application/json" \
  -d '{"email":"nobody@test.local","password":"wrong"}'
done
echo
```

Fichiers à toucher (reste)

- backend/init_db.py — enlever DROP, garder IF NOT EXISTS + seed admin
- frontend/src/app/login/login.ts — flag isLoading anti double-appel 2FA
- frontend/src/app/login/login.html — désactiver le bouton pendant l'envoi
- backend/app.py — cooldown / ne pas renvoyer 2 OTP (recommandé)
- frontend/src/app/app.routes.ts — ajouter canActivate: [authGuard]
- frontend/src/app/app.config.ts — withInterceptors([authInterceptor])
- (Optionnel) environment.ts, captcha dans login.ts

Ne pas refactorer tout le projet — finir ces points prioritaires d'abord.

Revue Logiqa · 29 juil. 2026 · ~70–80 % fait · DROP + 2FA double-envoi + guard + interceptor